

Class 7: Machine Learning 1

Julianne Lee (PID: A17979401)

Table of contents

Background	1
K-means clustering	1
Hierarchical Clustering	9
Principal Component Analysis (PCA)	11
Analysis of UK food data	11
Data import	11
Tidy the data	12
Exploratory analysis	12
PCA	12
PCA to the rescue	19

Background

Today we will explore some core machine learning methods that are very popular in bioinformatics. These include **clustering** and **dimensionality reduction**.

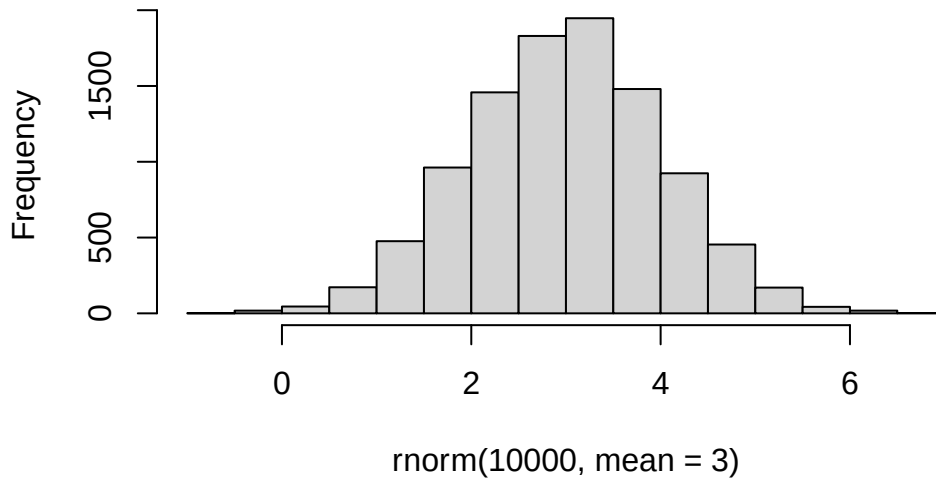
K-means clustering

The main function in “base” R for K-means clustering is called `kmeans()`

Before we go too deep let’s make up some “simple” data that we can cluster and know if we are getting a good answer or not. To do this, we can use `rnorm()` function:

```
hist(rnorm(10000, mean = 3))
```

Histogram of rnorm(10000, mean = 3)



```
x <- c(rnorm(30, -3), rnorm(30, 3))
x
```

```
[1] -2.9318982 -4.6792352 -1.6407664 -3.5181864 -3.0473506 -2.5458919
[7] -1.8979984 -2.0655714 -2.3523586 -2.7047803 -1.6434289 -3.1288289
[13] -2.1399915 -4.6735423 -1.4646020 -4.1300810 -3.8450625 -3.1673192
[19] -2.9874615 -3.1434974 -4.0921267  0.4893417 -3.5962298 -3.7681676
[25] -2.9333470 -2.3588798 -3.5204519 -2.2557804 -1.5808964 -3.9583128
[31]  2.2361458  3.3618336  5.3943202  2.5148298  4.5285810  2.8295923
[37]  2.9110366  2.5460797  3.1543237  1.4446500  3.2196664  1.9073120
[43]  0.8995827  2.6130943  1.7899030  4.3798697  3.7927521  1.2583539
[49]  2.3488206  1.8459600  2.6037731  2.3921440  2.7886683  3.1024160
[55]  1.2172253  3.8256302  2.5264254  3.7826068  4.1944998  4.5624670
```

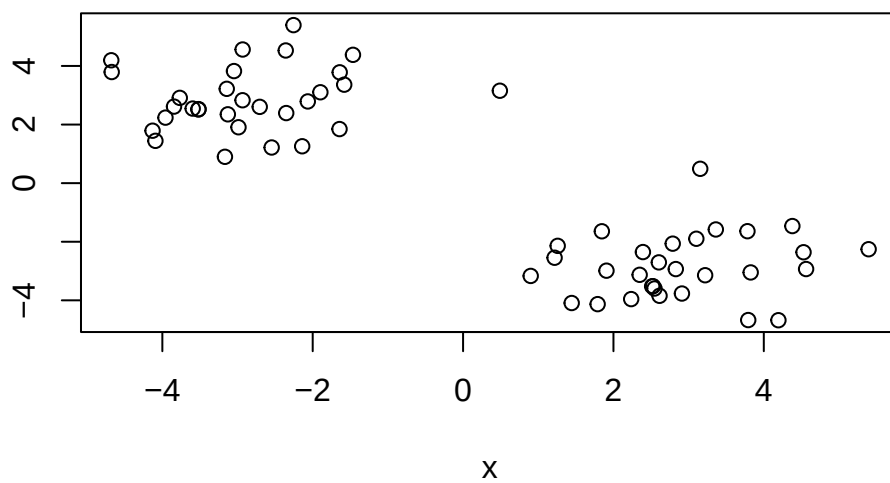
```
z <- cbind(x, rev(x))
z
```

```
      x
[1,] -2.9318982  4.5624670
[2,] -4.6792352  4.1944998
[3,] -1.6407664  3.7826068
[4,] -3.5181864  2.5264254
```

[5,]	-3.0473506	3.8256302
[6,]	-2.5458919	1.2172253
[7,]	-1.8979984	3.1024160
[8,]	-2.0655714	2.7886683
[9,]	-2.3523586	2.3921440
[10,]	-2.7047803	2.6037731
[11,]	-1.6434289	1.8459600
[12,]	-3.1288289	2.3488206
[13,]	-2.1399915	1.2583539
[14,]	-4.6735423	3.7927521
[15,]	-1.4646020	4.3798697
[16,]	-4.1300810	1.7899030
[17,]	-3.8450625	2.6130943
[18,]	-3.1673192	0.8995827
[19,]	-2.9874615	1.9073120
[20,]	-3.1434974	3.2196664
[21,]	-4.0921267	1.4446500
[22,]	0.4893417	3.1543237
[23,]	-3.5962298	2.5460797
[24,]	-3.7681676	2.9110366
[25,]	-2.9333470	2.8295923
[26,]	-2.3588798	4.5285810
[27,]	-3.5204519	2.5148298
[28,]	-2.2557804	5.3943202
[29,]	-1.5808964	3.3618336
[30,]	-3.9583128	2.2361458
[31,]	2.2361458	-3.9583128
[32,]	3.3618336	-1.5808964
[33,]	5.3943202	-2.2557804
[34,]	2.5148298	-3.5204519
[35,]	4.5285810	-2.3588798
[36,]	2.8295923	-2.9333470
[37,]	2.9110366	-3.7681676
[38,]	2.5460797	-3.5962298
[39,]	3.1543237	0.4893417
[40,]	1.4446500	-4.0921267
[41,]	3.2196664	-3.1434974
[42,]	1.9073120	-2.9874615
[43,]	0.8995827	-3.1673192
[44,]	2.6130943	-3.8450625
[45,]	1.7899030	-4.1300810
[46,]	4.3798697	-1.4646020
[47,]	3.7927521	-4.6735423

```
[48,] 1.2583539 -2.1399915
[49,] 2.3488206 -3.1288289
[50,] 1.8459600 -1.6434289
[51,] 2.6037731 -2.7047803
[52,] 2.3921440 -2.3523586
[53,] 2.7886683 -2.0655714
[54,] 3.1024160 -1.8979984
[55,] 1.2172253 -2.5458919
[56,] 3.8256302 -3.0473506
[57,] 2.5264254 -3.5181864
[58,] 3.7826068 -1.6407664
[59,] 4.1944998 -4.6792352
[60,] 4.5624670 -2.9318982
```

```
plot(z)
```



```
p <- 1:5
cbind(p, rev(p))
```

```
      p
[1,] 1 5
```

```
[2,] 2 4
[3,] 3 3
[4,] 4 2
[5,] 5 1
```

Now we can run `kmeans()` on this input `z` and see what the results look like.

```
kmeans(z, centers=2)
```

K-means clustering with 2 clusters of sizes 30, 30

Cluster means:

	x	
1	2.865752	-2.842757
2	-2.842757	2.865752

Clustering vector:

[illegible]

Within cluster sum of squares by cluster:

```
[1] 70.18275 70.18275
(between_SS / total_SS = 87.4 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

```
km <- kmeans(z,centers=2)
km
```

K-means clustering with 2 clusters of sizes 30, 30

Cluster means:

	x	
1	2.865752	-2.842757
2	-2.842757	2.865752

Clustering vector:

[illegible]

```
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

Within cluster sum of squares by cluster:

```
[1] 70.18275 70.18275
(between_SS / total_SS = 87.4 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"       "
```

```
attributes(km)
```

```
$names
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"       "
```

```
$class
[1] "kmeans"
```

Q. How many points are in each cluster?

```
km$size
```

```
[1] 30 30
```

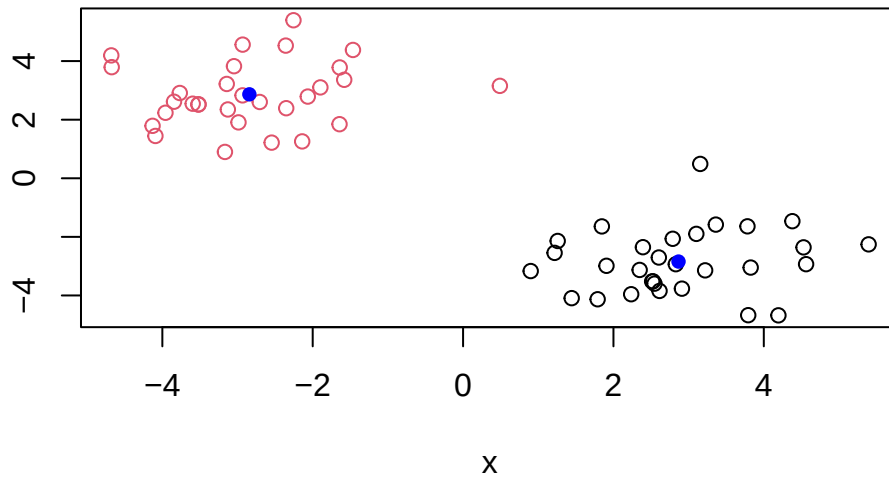
Q. What component of your result object details cluster assignment/membership?

```
km$centers
```

```
      x
1  2.865752 -2.842757
2 -2.842757  2.865752
```

Q. Plot z colored by the kmeans cluster assignment and add cluster centers as blue points

```
plot(z, col=km$cluster)+
  points(km$centers, col="blue", pch=16)
```



```
integer(0)
```

Q. Run a K-means clustering and plot the results asking for 4 clusters (K=4)

```
km2 <- kmeans(z, centers=4)
km2
```

K-means clustering with 4 clusters of sizes 20, 20, 10, 10

Cluster means:

```
      x
1  2.354592 -3.326415
2 -3.326415  2.354592
3 -1.875440  3.888072
4  3.888072 -1.875440
```

Clustering vector:

```
[1] 3 2 3 2 3 2 3 3 2 2 2 2 2 2 3 2 2 2 2 2 3 2 2 2 3 2 2 3 2 3 3 2 1 4 4 1 4 1 1 1
[39] 4 1 1 1 1 1 1 4 1 1 1 1 1 1 4 4 1 4 1 4 1 4
```

Within cluster sum of squares by cluster:

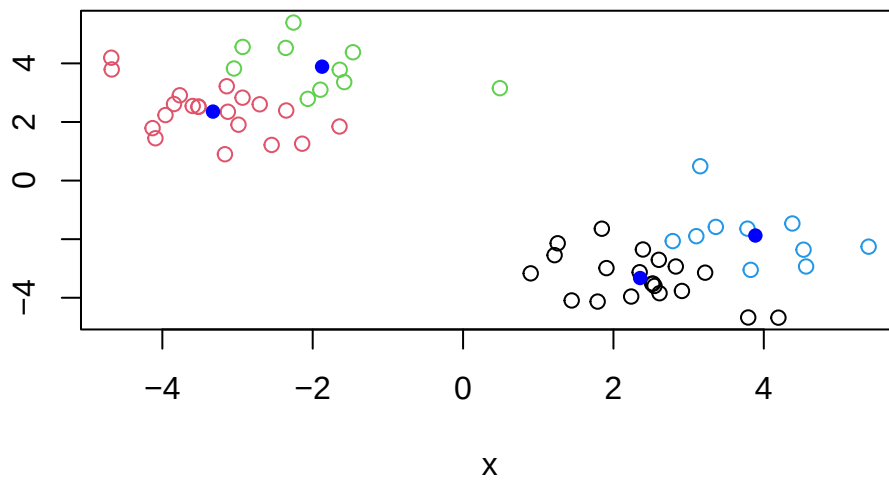
```
[1] 25.63087 25.63087 14.83931 14.83931
```

```
(between_SS / total_SS = 92.8 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

```
plot(z,col=km2$cluster)+
  points(km2$centers, col="blue", pch=16)
```



```
integer(0)
```

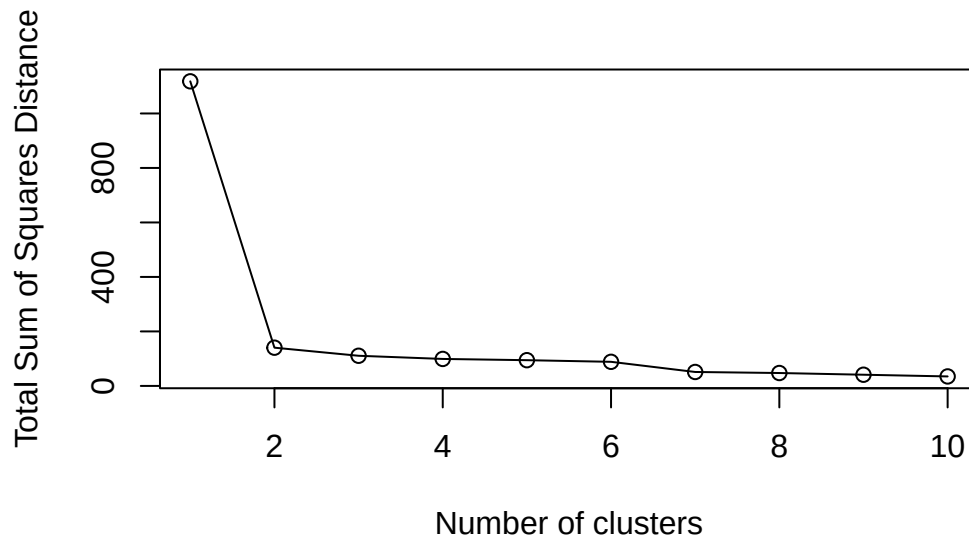
N.B. You need to tell K-means the number of clusters (i.e. set `centers=2`)!!

One approach is to try different values for `centers` and then pick the best...

```
ans <- NULL
for (i in 1:10) {
  km <- kmeans(z, centers=i)
  ans <- c(ans, km$tot.withinss)
}
```



```
plot(ans,typ="o",
      xlab="Number of clusters",
      ylab="Total Sum of Squares Distance")
```



Hierarchical Clustering

The main function in “base” R for Hierarchical Clustering is called `hclust()`

This function does not take your “raw” data for clustering. You must first build a “distance matrix” from your data and pass this as input to `hclust()`

```
d <- dist(z)
hc <- hclust(d)
hc
```

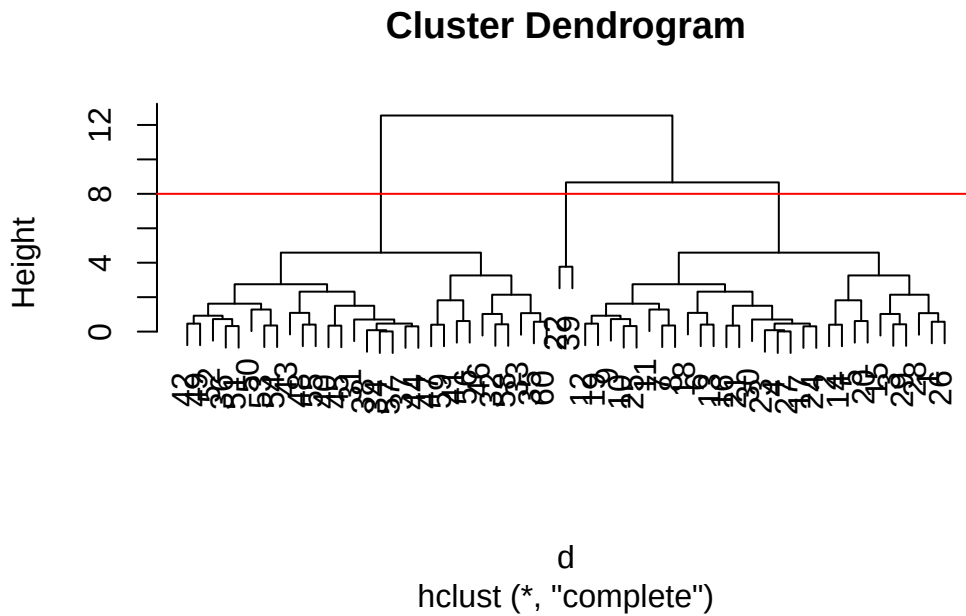
Call:

```
hclust(d = d)
```

```
Cluster method : complete
Distance       : euclidean
Number of objects: 60
```

There is a bespoke `plot()` method for `hclust()` result objects.

```
plot(hc)+
abline(h=8, col="red")
```



```
integer(0)
```

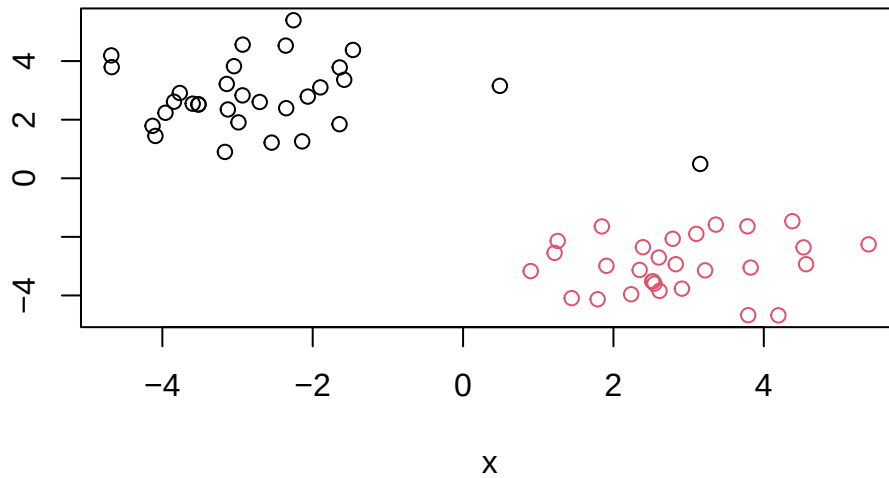
Once we have our `hclust` object (our “tree” of “cluster dendrogram”) we can “*cut*” the tree to reveal the clustering pattern.

```
cutree(hc, k=4)
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 3 4 4 3 4 3 3 3
[39] 2 3 4 3 3 3 3 4 4 3 3 3 3 3 3 3 4 3 4 4 4
```

Q. Make a plot of `z` with your `hclust` results (i.e. colored by cluster membership)

```
grps <- cutree(hc, k=2)
plot(z, col=grps)
```



Principal Component Analysis (PCA)

PCA is a dimensionality reduction method that is popular for revealing patterns in complex datasets.

Analysis of UK food data

Let's look at some data on the eating habits of folks from the UK to see if there are patterns and trends that have some regions being distinct from others.

Data import

The data is made available in CSV format so we can use the `read.csv()` function for import to R:

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
x
```

		X	England	Wales	Scotland	N.Ireland
1	Cheese		105	103	103	66
2	Carcass_meat		245	227	242	267
3	Other_meat		685	803	750	586
4	Fish		147	160	122	93
5	Fats_and_oils		193	235	184	209
6	Sugars		156	175	147	139
7	Fresh_potatoes		720	874	566	1033
8	Fresh_Veg		253	265	171	143
9	Other_Veg		488	570	418	355
10	Processed_potatoes		198	203	220	187
11	Processed_Veg		360	365	337	334
12	Fresh_fruit		1102	1137	957	674
13	Cereals		1472	1582	1462	1494
14	Beverages		57	73	53	47
15	Soft_drinks		1374	1256	1572	1506
16	Alcoholic_drinks		375	475	458	135
17	Confectionery		54	64	62	41

Tidy the data

Fix anything that went wrong with data import.

Exploratory analysis

Make some plots to help make sense of obvious trends..

PCA

Q1. How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

```
dim(x)
```

```
[1] 17  5
```

```
head(x)
```

	X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66
2	Carcass_meat	245	227	242	267
3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139

```
rownames(x) <- x[,1]
x <- x[,-1]
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

```
dim(x)
```

```
[1] 17 4
```

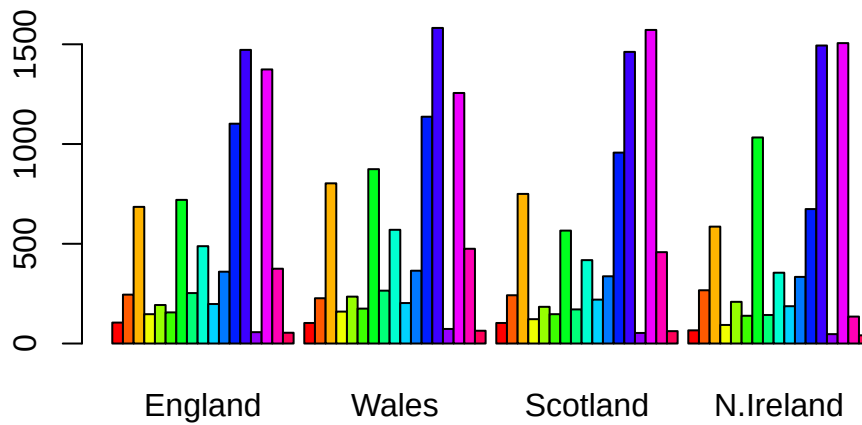
```
x <- read.csv(url, row.names=1)
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

Q2. Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

I prefer the second since the first approach keeps deleting the first row name if we run the code again.

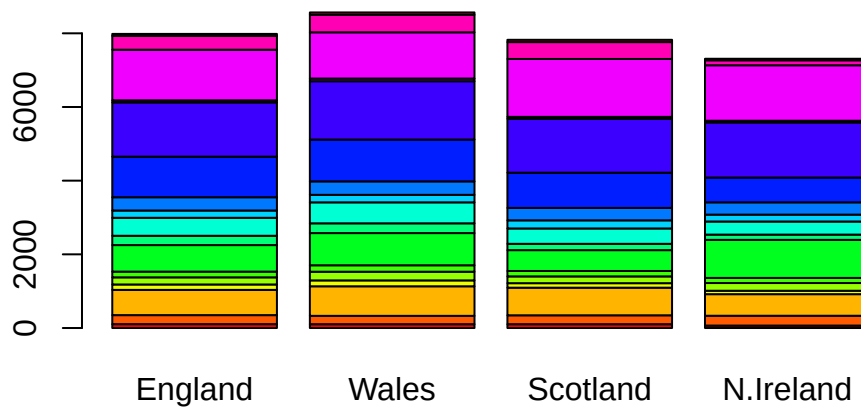
```
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```



Q3. Changing what optional argument in the above `barplot()` function results in the following plot?

`beside=...`

```
barplot(as.matrix(x), beside=F, col=rainbow(nrow(x)))
```



```
library(tidyr)

x_long <- x |>
  tibble::rownames_to_column("Food") |>
  pivot_longer(cols = -Food,
               names_to = "Country",
               values_to = "Consumption")

dim(x_long)
```

```
[1] 68  3
```

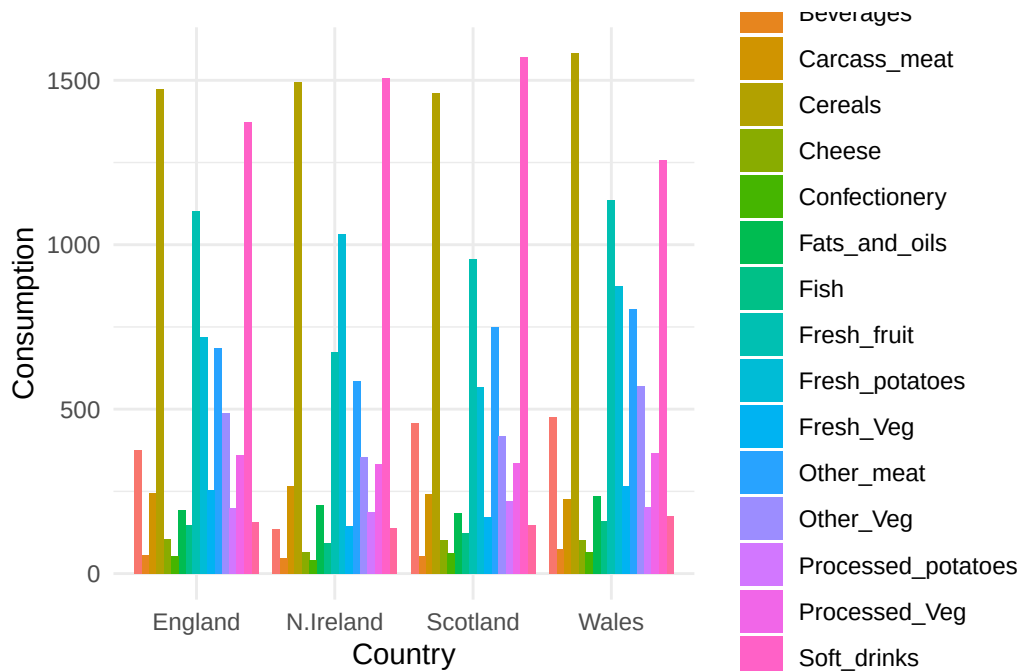
```
head(x_long)
```

```
# A tibble: 6 x 3
  Food      Country Consumption
<chr>    <chr>         <int>
1 "Cheese" England         105
2 "Cheese" Wales           103
3 "Cheese" Scotland        103
4 "Cheese" N.Ireland          66
```

```
5 "Carcass_meat " England          245
6 "Carcass_meat " Wales            227
```

```
library(ggplot2)

ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_minimal()
```



Q4. Changing what optional argument in the above `ggplot()` code results in a stacked barplot figure?

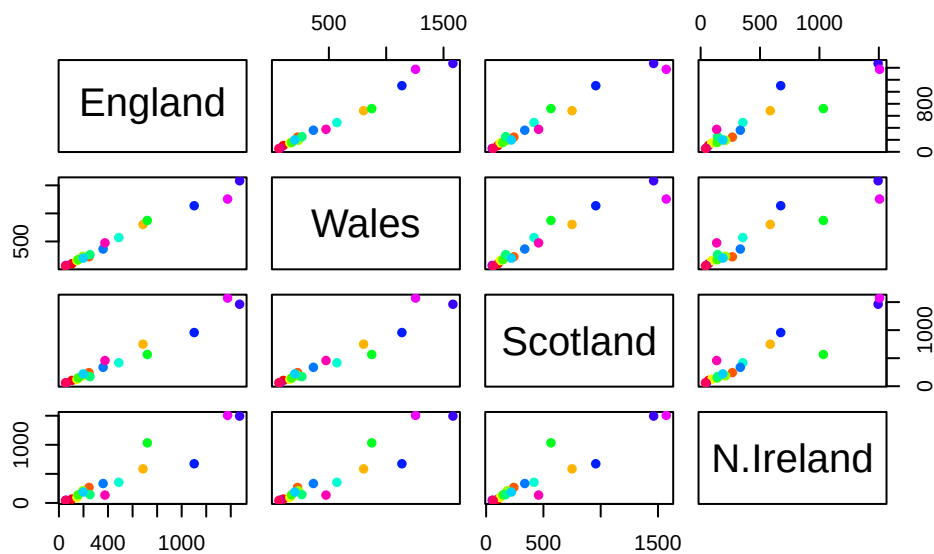
`geom_col(position=...)`

```
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "stack") +
  theme_minimal()
```



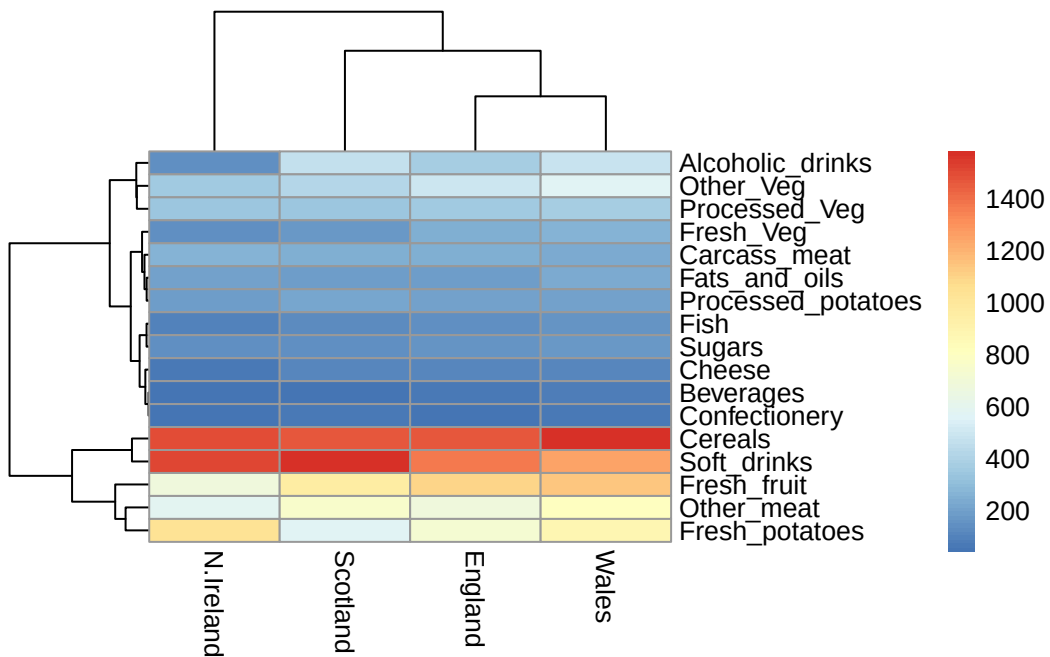

Q5. We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



```
library(pheatmap)

pheatmap( as.matrix(x) )
```



Key-point: Even relatively small datasets can prove challenging to interpret.

PCA to the rescue

The main function in “base” R for PCA is called `prcomp()`. This function wants the “observations” to be rows and the “variables” to be columns.

So here we need to take the transpose of our `x` input object.

```
pca <- prcomp(t(x))
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	2.921e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

The returned `pca` object has components that we can use to make our main result figures:

```
attributes(pca)
```

\$names

```
[1] "sdev"      "rotation" "center"    "scale"     "x"
```

\$class

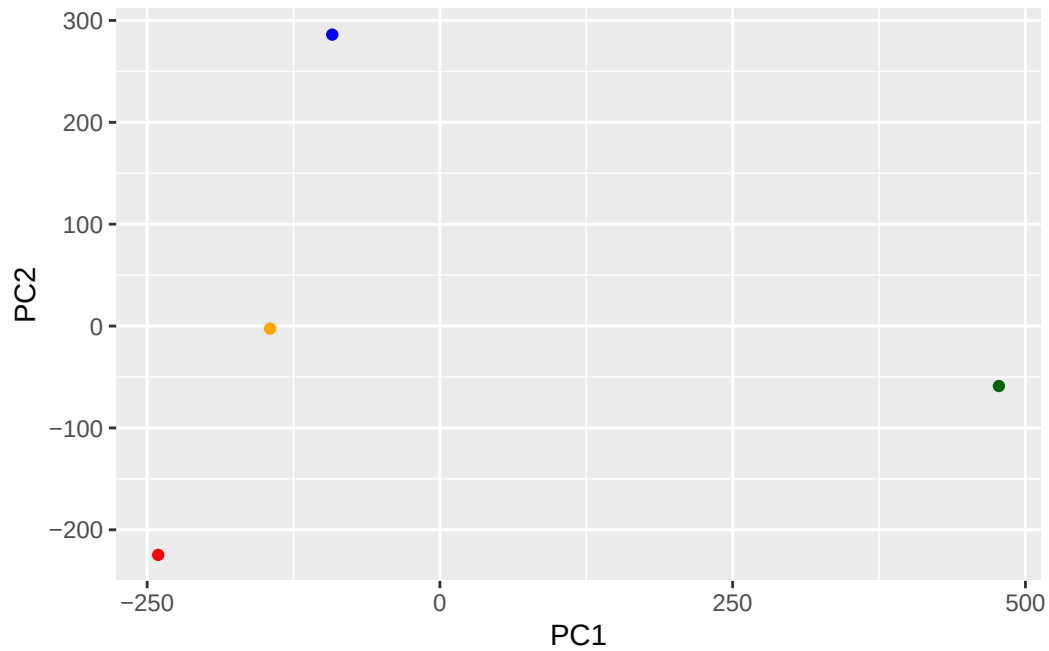
```
[1] "prcomp"
```

The main result figure from this analysis is called a “**PC score plot**” (a.k.a. an “ordination plot” “PC plot” or “PC1 vs PC2 plot”).

This plot shows how samples (in this case countries) relate to each other along our new PC axis.

```
mycols <- c("orange", "red", "blue", "darkgreen")

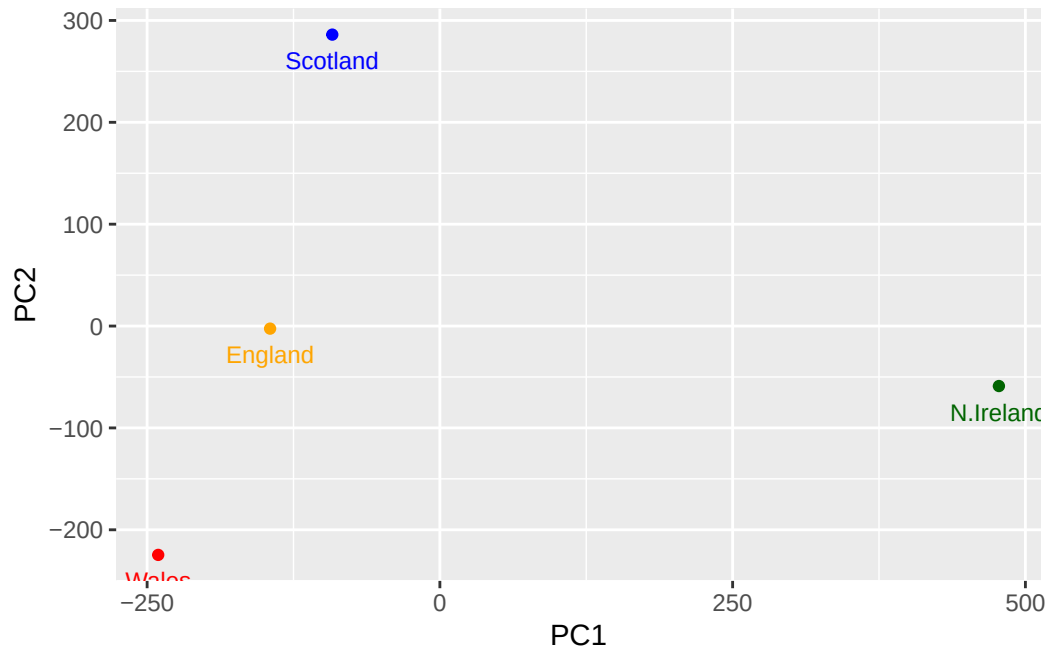
ggplot(pca$x) +
  aes(x = PC1, y = PC2) +
  geom_point(col=mycols)
```



```
pca$x
```

	PC1	PC2	PC3	PC4
England	-144.99315	-2.532999	105.768945	-9.152022e-15
Wales	-240.52915	-224.646925	-56.475555	5.560040e-13
Scotland	-91.86934	286.081786	-44.415495	-6.638419e-13
N.Ireland	477.39164	-58.901862	-4.877895	1.329771e-13

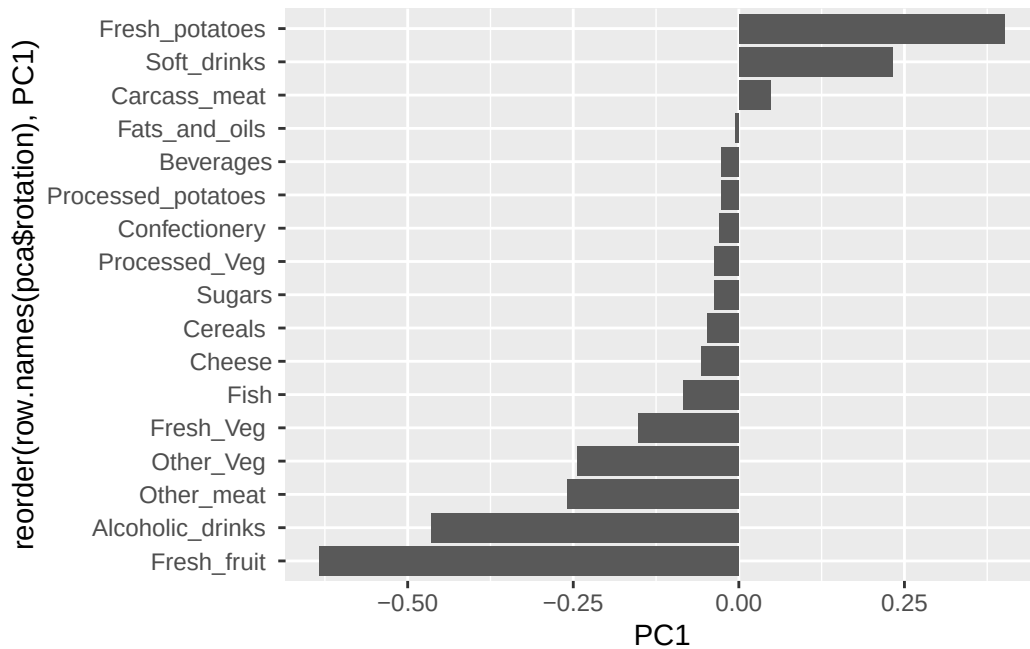
```
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label=row.names(pca$x)) +
  geom_point(col=mycols) +
  geom_text(size=3, vjust=2, col=mycols)
```



```
pca$x
```

	PC1	PC2	PC3	PC4
England	-144.99315	-2.532999	105.768945	-9.152022e-15
Wales	-240.52915	-224.646925	-56.475555	5.560040e-13
Scotland	-91.86934	286.081786	-44.415495	-6.638419e-13
N.Ireland	477.39164	-58.901862	-4.877895	1.329771e-13

```
ggplot(pca$rotation) +
  aes(PC1, reorder(row.names(pca$rotation), PC1))+
  geom_col()
```



Q9. Generate a similar ‘loadings plot’ for PC2. What two food groups feature prominently and what does PC2 mainly tell us about?

Soft drinks and alcoholic drinks are prominent. PC2 mainly tells about a beverage consumption among countries in how much they consume the soft drinks and alcoholic drinks relative to the other foods.

```
ggplot(pca$rotation) +
  aes(PC2, reorder(row.names(pca$rotation), PC2))+
  geom_col()
```

